An Extended Kalman Filter for Skid-Steer Robot Odometry with Online Effective Wheel Separation Estimation

Stuart G. Johnson ChatGPT (OpenAI)

July 13, 2026

Abstract

This note derives an Extended Kalman Filter (EKF) for odometry estimation of a skid-steer robot modeled approximately as a differential-drive robot. The filter estimates planar pose, gyroscope bias, and an effective wheel separation. The effective wheel separation is not interpreted as a purely geometric quantity; rather, it is a compact parameter that absorbs some of the mismatch between ideal differential-drive kinematics and the actual skid-steer motion.

1 Motivation

A four-wheel skid-steer robot is often modeled as a differential-drive robot. This approximation is convenient, but it is imperfect. During turns, the wheels scrub against the ground. This violates the ideal nonholonomic differential-drive model.

The goal of this filter is to provide proprioceptive odometry suitable for ROS2 navigation. The odometry will drift over time, but it should be locally smooth and track true robot motion. The filter uses wheel velocity estimates and gyroscope measurements. It also estimates an effective wheel separation in order to adapt the differential-drive model to the observed robot behavior.

2 State Vector

The EKF state at time k is

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \\ \theta_k \\ b_{g,k} \\ b_k \end{bmatrix}. \tag{1}$$

The state components are:

- x_k, y_k: robot position in the odometry frame,
- θ_k: robot heading,
- $b_{g,k}$: gyroscope yaw-rate bias,
- b_k: effective wheel separation.

The effective wheel separation b_k should not be interpreted too literally as the physical distance between wheels. For a skid-steer robot, it is an effective kinematic parameter that can absorb wheel slip, tire compliance, load transfer, and other unmodeled effects.

3 Inputs and Measured Quantities

Let the measured left and right wheel linear velocities be

$$\mathbf{u}_k = \begin{bmatrix} v_{L,k} \\ v_{R,k} \end{bmatrix}. \tag{2}$$

The differential-drive forward velocity estimate is

$$v_k = \frac{v_{R,k} + v_{L,k}}{2}. \tag{3}$$

The wheel-derived yaw-rate estimate is

$$\omega_k = \frac{v_{R,k} - v_{L,k}}{b_k}. \tag{4}$$

The gyroscope measurement is denoted

$$z_{g,k}. \tag{5}$$

For this EKF, the wheel and gyroscope data are averaged over common robot-side sampling intervals and then emitted to the ROS2 system. Thus, the filter should process synchronized wheel and IMU packets as one discrete update step.

4 Continuous-Time Process Model

The continuous-time kinematic model is

$$\dot{x} = v \cos\theta, \tag{6}$$
$$\dot{y} = v \sin\theta, \tag{7}$$
$$\dot{\theta} = \frac{v_R - v_L}{b}, \tag{8}$$
$$\dot{b}_g = 0, \tag{9}$$
$$\dot{b} = 0. \tag{10}$$

5 Discretization

The Euler update process model is

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k \tag{11}$$

where $\mathbf{w}_k$ is the process noise. f is defined as:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) \tag{12}$$
$$x_{k+1} = x_k + \Delta t_k v_k \cos\theta_k, \tag{13}$$
$$y_{k+1} = y_k + \Delta t_k v_k \sin\theta_k, \tag{14}$$
$$\theta_{k+1} = \theta_k + \Delta t_k \omega_k, \tag{15}$$
$$b_{g,k+1} = b_{g,k}, \tag{16}$$
$$b_{k+1} = b_k. \tag{17}$$

6 Process Jacobian

The EKF process Jacobian is

$$F_k = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{k|k},\mathbf{u}_k}. \tag{18}$$

The full state Jacobian is

$$F_k = \begin{bmatrix} 1 & 0 & -\Delta t_k v_k sin\theta_k & 0 & 0 \\ 0 & 1 & \Delta t_k v_k cos\theta_k & 0 & 0 \\ 0 & 0 & 1 & 0 & -\Delta t_k \frac{\omega_k}{b_k} \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}. \tag{19}$$

7 Process Noise

Let the wheel speed covariance be

$$\Sigma_u = \begin{bmatrix} \sigma_{v_L}^2 & \sigma_{v_L v_R} \\ \sigma_{v_L v_R} & \sigma_{v_R}^2 \end{bmatrix}. \tag{20}$$

If the left and right wheel speed errors are assumed uncorrelated, then

$$\Sigma_u = \begin{bmatrix} \sigma_{v_L}^2 & 0 \\ 0 & \sigma_{v_R}^2 \end{bmatrix}. \tag{21}$$

The control Jacobian is

$$G_k = \left.\frac{\partial f}{\partial \mathbf{u}}\right|_{\hat{\mathbf{x}}_{k|k},\mathbf{u}_k}. \tag{22}$$

which evaluates to:

$$G_k = \begin{bmatrix} \Delta t_k \frac{1}{2} cos\theta_k & \Delta t_k \frac{1}{2} cos\theta_k) \\ \Delta t_k \frac{1}{2} sin\theta_k & \Delta t_k \frac{1}{2} sin\theta_k \\ -\frac{\Delta t_k}{b_k} & \frac{\Delta t_k}{b_k} \\ 0 & 0 \\ 0 & 0 \end{bmatrix}. \tag{23}$$

Propagation of errors therefore gives a contribution to process covariance of

$$Q_{u,k} = G_k \Sigma_u G_k^T \tag{24}$$

Random-walk process noise for gyroscope bias and effective wheel separation is

$$Q_{b,k} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \sigma_{bg}^2 \Delta t_k & 0 \\ 0 & 0 & 0 & 0 & \sigma_b^2 \Delta t_k \end{bmatrix}. \tag{25}$$

The skid-steer kinematic model noise is modeled as a constant yaw-rate error over the sampling interval. This results in an additive term in the yaw state diagonal of the process covariance:

$$Q_{m,k} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \sigma_\omega^2 \Delta t_k^2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}. \tag{26}$$

Thus the total process covariance contribution is

$$Q_k = Q_{u,k} + Q_{b,k} + Q_{m,k} \tag{27}$$

8 Gyroscope Measurement Model

The gyroscope measurements return yaw rate plus bias:

$$z_{g,k} = \omega_k + b_{g,k} + \nu_k, \tag{28}$$

where ν_k is zero-mean gyroscope measurement noise. The measurement function is

$$h(\mathbf{x}_k, \mathbf{u}_k) = \frac{v_{R,k} - v_{L,k}}{b_k} + b_{g,k}. \tag{29}$$

The measurement Jacobian is

$$H_k = \left.\frac{\partial h}{\partial \mathbf{x}}\right|_{\hat{\mathbf{x}}_{k+1|k}, \mathbf{u}_k}. \tag{30}$$

Explicitly,

$$H_k = \begin{bmatrix} 0 & 0 & 0 & 1 & -\frac{v_{R,k} - v_{L,k}}{b_k^2} \end{bmatrix}. \tag{31}$$

The gyroscope measurement covariance is

$$R = \sigma_g^2. \tag{32}$$

9 EKF Algorithm

At each synchronized measurement, the EKF performs the following operations:

9.1 Prediction

Given the posterior estimate from the previous step,

$$\hat{\mathbf{x}}_{k|k}, \tag{33}$$

and covariance

$$P_{k|k}, \tag{34}$$

compute

$$\hat{\mathbf{x}}_{k+1|k} = f(\hat{\mathbf{x}}_{k|k}, \mathbf{u}_k). \tag{35}$$

Compute F_k and $G_{u,k}$ at $\hat{\mathbf{x}}_{k|k}$ and $\mathbf{u}_k$. Then propagate covariance:

$$P_{k+1|k} = F_k P_{k|k} F_k^T + Q_k. \tag{36}$$

9.2 Update

The gyroscope innovation is

$$r_k = z_k - h(\hat{\mathbf{x}}_{k+1|k}, \mathbf{u}_k). \tag{37}$$

The innovation covariance is

$$S_k = H_k P_{k+1|k} H_k^T + R. \tag{38}$$

The Kalman gain is

$$K_{g,k} = P_{k+1|k} H_{g,k}^T S_{g,k}^{-1}. \tag{39}$$

The posterior state estimate is

$$\hat{\mathbf{x}}_{k+1|k+1} = \hat{\mathbf{x}}_{k+1|k} + K_{g,k} r_{g,k}. \tag{40}$$

The covariance update is:

$$P_{k+1|k+1} = (I - K_k H_k) P_{k+1|k} \tag{41}$$

10 ROS2 Odometry Covariances

The ROS2 `nav_msgs/Odometry` message contains pose and twist covariances.

The pose is expressed in the frame named by `header.frame_id`, typically `odom`. The twist is expressed in the frame named by `child_frame_id`, typically `base_link`.

10.1 Pose Covariance

The planar pose covariance is obtained directly from the EKF covariance matrix. The entries corresponding to x, y, and yaw are inserted into the ROS2 6×6 pose covariance matrix with ordering

$$[x,\ y,\ z,\ roll,\ pitch,\ yaw]. \tag{42}$$

For example,

$$\Sigma_{pose}(x, x) = P(x, x), \tag{43}$$
$$\Sigma_{pose}(y, y) = P(y, y), \tag{44}$$
$$\Sigma_{pose}(yaw, yaw) = P(\theta, \theta), \tag{45}$$
$$\Sigma_{pose}(x, yaw) = P(x, \theta), \tag{46}$$
$$\Sigma_{pose}(y, yaw) = P(y, \theta). \tag{47}$$

10.2 Twist Covariance

The state definition does not include the twist. Therefore twist covariance must be derived from the uncertainty in the reported twist estimate.

The twist estimate returned to ROS2 is:

$$v = \frac{v_L + v_R}{2}, \tag{48}$$
$$\omega = \frac{v_R - v_L}{b}, \tag{49}$$

The ROS2 twist covariance uses ordering

$$[v_x,\ v_y,\ v_z,\ \omega_x,\ \omega_y,\ \omega_z]. \tag{50}$$

Twist covariance for the non-zero components (v_x, ω_z) is obtained by propagation of errors:

$$\Sigma_{\text{twist}}(v_x, \omega_z) \approx J \begin{bmatrix} \sigma_{v_L}^2 & 0 & 0 \\ 0 & \sigma_{v_R}^2 & 0 \\ 0 & 0 & \sigma_b^2 \end{bmatrix} J^T \tag{51}$$

where:

$$J = \begin{bmatrix} \frac{\partial v}{\partial v_L} & \frac{\partial v}{\partial v_R} & \frac{\partial v}{\partial b} \\ \frac{\partial \omega}{\partial v_L} & \frac{\partial \omega}{\partial v_R} & \frac{\partial \omega}{\partial b} \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ -\frac{1}{b} & \frac{1}{b} & -\frac{(v_R - v_L)}{b^2} \end{bmatrix} \tag{52}$$

Note that $\Sigma_{\text{twist}}(v_x, \omega_z)$ is a 2x2 matrix which must be distributed into the twist covariance.

ROS2 does not generally like covariance diagonals which are zero. For twist and state covariance, otherwise zero diagonals are set to 9999.0.